SelVeri User Manual

Current Source Language and Runtime Functionality

SelVeri Project

June 2026

Contents

1 Overview

SelVeri is an educational imperative programming language with built-in runtime verification. A SelVeri program is written in a `.svi` source file, compiled to the SelVerIR intermediate representation, and interpreted by the SelVerIR abstract machine. During interpretation, specifications embedded in the source program are checked by the verification engine.

The current implementation supports:

- statically declared `Int`, `Real`, and multidimensional `List` variables;
- arithmetic and boolean expressions;
- declarations, assignments, list element assignments, `if`, `while`, and `pass`;
- functions with parameters, return values, and the special return variable `retvar`;
- console input and output with `read()`, `write()`, and `writeline()`;
- embedded first-order, past temporal, future temporal, and separated temporal specifications;
- witness extraction through `obtain`;
- structured diagnostics for parse, compile, runtime, and verification errors.

2 Running Programs

Install the project dependencies from the repository root:

```
pip install -r requirements.txt
pip install -e .
```

Run a SelVeri source file:

```
selveri examples/example_loop.svi
```

The same pipeline can be launched through Python:

```
python -m selveri examples/example_loop.svi
```

To write generated SelVerIR to disk, pass `-emit-ir`. If no output path is supplied, the IR file is written next to the source with the `.svir` extension.

```
selveri examples/example_loop.svi --emit-ir
selveri examples/example_loop.svi --emit-ir --output-ir out/example_loop.svir
```

Useful command-line flags are:

Flag	Meaning
`-emit-ir`	write generated SelVerIR to a file
`-output-ir PATH`	choose the generated IR path
`-print-ir`	print generated SelVerIR before execution
`-max-steps N`	stop after at most `N` interpreted instructions
`-debug`	print compilation time, execution time, final state, stack, and steps

Future-time temporal specifications use MONA through the verification engine. For formulas using `Next`, `Eventually`, `Always`, or `Until`, the `mona` executable must be available on `PATH`.

3 Lexical Structure

Whitespace is ignored except where it separates tokens. SelVeri supports line comments and block comments:

```
// line comment
/* block comment */
```

Identifiers may start with a Latin or Greek letter and may continue with letters, digits, underscores, and Greek characters. In source programs, identifiers do not start with an underscore in the high-level grammar. Specification-bound identifiers may also start with an underscore.

Integer literals contain only digits. Real literals contain digits, a decimal point, and more digits:

```
0
42
3.14159
```

The names `retvar`, `read`, `write`, `writeline`, and `len` are reserved and cannot be declared by user programs.

4 Types and Values

4.1 Scalar Types

`Int` stores integer values. `Real` stores floating-point real values. Integer values can be assigned to `Real` variables, but real values cannot be assigned to `Int` variables.

```
x : Int;
x := 10;

y : Real;
y := 3.5;
y := x;      // allowed: Int to Real
```

Variables are initialized when declared. An `Int` starts as `0`; a `Real` starts as `0.0`.

4.2 Lists

A list type has an element type, a rank, and one shape expression per dimension:

```
numbers : List[Int, 1, 5];
matrix  : List[Real, 2, 3, 4];
```

`List[Int, 1, 5]` is a one-dimensional list of five integers. `List[Real, 2, 3, 4]` is a two-dimensional real list with shape 3×4. The rank must match the number of shape expressions.

List literals use square brackets and can be nested:

```
numbers := [1, 2, 3, 4, 5];
```

```
matrix : List[Int, 2, 3, 2];
matrix := [[1, 2], [3, 4], [5, 6]];
```

Lists are stored by the runtime as flat arrays, but source code uses multidimensional indexing:

```
matrix[1] := [30, 40];  // assign a sub-list
matrix[2][1] := 8;      // assign one scalar element
```

Indexes are zero-based. Out-of-bounds accesses are runtime errors. The expression `len(xs)` returns the first-dimension length of a list.

5 Expressions

5.1 Arithmetic Expressions

Arithmetic expressions support:

- literals and variables;
- list indexing;
- `len(name)`;
- unary minus;
- `+`, `-`, `*`, and `/`;
- `read()`;
- `obtain(...)`.

Operator precedence follows the usual arithmetic order: unary minus, multiplication and division, then addition and subtraction. Parentheses override precedence.

```
x := 1 + 2 * 3;
y := (1 + 2) * 3;
z := -x + y / 2;
```

Integer division is used when the expression type is `Int`; real division is used when the expression type is `Real`. Division by zero is a runtime error.

5.2 Boolean Expressions

Boolean expressions support:

- `true` and `false`;
- comparisons `=`, `<`, `<=`, `>`, and `>=`;
- `not`, `and`, `xor`, and `or`;
- arithmetic expressions used as truthy values;
- specification blocks used as boolean predicates.

```
if x > 0 and not (y = 0) then
    writeline(x);
fi
```

The boolean precedence order is `not`, `and`, `xor`, then `or`. Arithmetic expressions are false when they evaluate to zero and true otherwise.

6 Statements

Statements that must end with a semicolon are declarations, assignments, list assignments, function-call statements, `pass`, `write`, `writeline`, `return`, and specification annotations. `if ... fi` and `while ... od` must not be followed by a semicolon.

6.1 Declarations and Assignments

```
x : Int;
x := 1;

r : Real;
r := 2.5;

xs : List[Int, 1, 3];
xs := [10, 20, 30];
xs[1] := 99;
```

A name may be declared only once in the same scope. Inner block scopes may declare names that hide outer declarations. Assigning to a name found in an outer scope updates that outer variable.

6.2 Conditionals

```
if x = 1 then
    x := 2;
else
    x := 3;
fi
```

The `else` part is optional:

```
if x > 0 then
    writeline(x);
fi
```

Each branch executes in a child scope.

6.3 Loops

```
i : Int;
i := 0;
sum : Int;
sum := 0;

while i < len(xs) do
    sum := sum + xs[i];
    i := i + 1;
od
```

Each loop body execution uses a child scope. Variables declared inside a loop body are not visible after the body scope exits. The interpreter enforces a maximum step count; use `-max-steps` to change it.

6.4 Pass

`pass;` performs no operation.

```
if x = 0 then
    pass;
else
    x := x - 1;
fi
```

7 Input and Output

`write(expr)` prints a value without a trailing newline. `writeline(expr)` prints a value followed by a newline. Lists are printed in bracketed form.

```
x : Int;
x := 5;
write(x);
writeline(2);
```

`read()` reads one line from standard input and parses it as an integer or real scalar.

```
z : Int;
z := read();
writeline(z);

w : Real;
w := read();
writeline(w);
```

8 Functions

Function declarations must appear before the top-level statement sequence:

```
function add(a: Int, b: Int) -> Int ::
    return a + b;
end

result : Int;
call add(2, 3);
result := retvar;
```

A function call is a statement of the form `call name(args);`. The return value is written to the special variable `retvar` in the caller. A program normally copies `retvar` into a declared variable immediately after the call.

8.1 Parameters and Return Types

Function parameters and return values can use scalar types, fully shaped list types, and rank-1 dynamic list types:

```
function sumFive(xs: List[Int, 1, 5]) -> Int ::
    i : Int;
    i := 0;
    total : Int;
    total := 0;
    while i < 5 do
        total := total + xs[i];
        i := i + 1;
    od
    return total;
end

function sum(xs: List[Int, 1]) -> Int ::
    i : Int;
    i := 0;
    total : Int;
    total := 0;
    while i < len(xs) do
        total := total + xs[i];
        i := i + 1;
    od
    return total;
end
```

`List[Int, 1]` in a function type is a rank-1 variable-length list. In the current compiler, dynamic list parameters are supported only for rank 1.

If a function body does not guarantee a `return` on every path, the compiler appends a default return. The default is `0` for `Int`, `0.0` for `Real`, and a zero-filled list for list return types.

9 Specifications

Specification annotations are written in braces and end with semicolons when used as statements:

```
{ x > 0 };
{ Forall i in [0...len(xs) - 1] . xs[&i] >= 0 };
```

The preprocessor extracts specification blocks before normal parsing. Nested specification braces are not allowed. Comments may appear inside specification blocks.

9.1 State Formulas

Specifications can contain boolean expressions, logical connectives, implication, and quantifiers.

Syntax	Meaning
`! p`	negation
`p && q`	conjunction
`p || q`	disjunction
`p => q`	implication, right-associative
`Forall x in D . p`	universal quantification
`Exists x in D . p`	existential quantification

The symbolic forms ∀, ∃, and ∈ are accepted by the specification grammar as alternatives to `Forall`, `Exists`, and `in`.

Bound variables are referenced with an ampersand:

```
{ Forall i in [0...4] . xs[&i] > 0 };
{ Exists e in xs . &e = 10 };
```

9.2 Quantifier Domains

Supported quantifier domains are:

- explicit values: `[a, b, c]`;
- integer ranges: `[lo...hi]`;
- real intervals: `[lo; hi]`, `(lo; hi)`, `[lo; hi)`, and `(lo; hi]`;
- an identifier, commonly a list name;
- scalar types: `Int` and `Real`;
- visible variables of a scalar type: `Var[Int]` and `Var[Real]`.

```
{ Forall i in [0...len(xs) - 1] . xs[&i] >= 0 };
{ Exists v in [1, 2, 3] . &v = 2 };
{ Forall x in Var[Int] . &x >= 0 };
```

9.3 Temporal Specifications

SelVeri supports past-time, future-time, and separated temporal formulas.

Past-time operators:

Operator	Meaning
`Previously p`	`p` held at the previous step
`Once p`	`p` held at some past or current step
`Historically p`	`p` held at every past and current step
`p Since q`	`q` held at some past point and `p` held since then

Future-time operators:

Operator	Meaning
`Next p`	`p` holds at the next step
`Eventually p`	`p` holds at some future or current step
`Always p`	`p` holds at every future and current step
`p Until q`	`q` eventually holds and `p` holds until then

Examples:

```
x : Int;
x := 0;
{ Next x = 1 };
x := 1;

{ Eventually x = 3 };
x := 2;
x := 3;

{ Historically x >= 0 };
```

A separated temporal formula has a past-time antecedent and a future-time consequent:

```
{ (Once x = 0) => (Eventually x = 5) };
```

9.4 Named Specifications and Temporal Regions

A named specification assigns a formula to a name:

```
{ nonnegative := Always x >= 0 };
```

Future temporal and separated temporal specifications can be delimited with an end marker:

```
{ target := Eventually x = 10 };
x := 5;
x := 10;
{ end target };
```

Past temporal and separated temporal specifications can be delimited with a start marker:

```
{ start history };
x := 1;
x := 2;
{ history := Once x = 1 };
```

The compiler enforces the marker direction:

- `{ start name }` applies only to past temporal or separated formulas.
- `{ end name }` applies only to future temporal or separated formulas.
- A started named specification must be defined in the same scope.
- A future end marker must refer to a named specification already defined in the same scope.

9.5 Specification Predicates in Boolean Expressions

A specification block can be used where a boolean expression is expected. At runtime it is checked by the verifier and produces `1` for true or `0` for false.

```
if { Exists i in [0...4] . xs[&i] = 10 } then
    writeline(1);
else
    writeline(0);
fi
```

If the interpreter is run without a verifier, specification predicates are treated optimistically as true. The standard `selveri` command attaches a verifier.

10 Witness Extraction with `obtain`

`obtain` asks the verifier for a witness value satisfying an existential specification. The first argument names the bound variable whose witness should be returned.

```
y : Int;
y := 5;

r : Int;
r := obtain(&x, { Exists x in Int . &x = y });
{ r = 5 };
```

The result can be used anywhere an arithmetic expression is accepted:

```
r2 : Int;
r2 := obtain(&a, { Exists a in [1...10] . &a * &a = 9 });

if obtain(&i, { Exists i in [2...5] . &i = 2 }) - 2 then
    writeline(1);
else
    writeline(0);
fi
```

If no verifier is attached, `obtain` is a runtime error. If the formula has no suitable witness, verification fails.

11 Complete Example

```
function sum(xs: List[Int, 1]) -> Int ::
    i : Int;
    i := 0;
    total : Int;
    total := 0;

    while i < len(xs) do
        total := total + xs[i];
```

```
        i := i + 1;
    od

    return total;
end

xs : List[Int, 1, 5];
xs := [1, 2, 3, 4, 5];

{ Forall i in [0...len(xs) - 1] . xs[&i] > 0 };

call sum(xs);
answer : Int;
answer := retvar;

{ answer = 15 };
writeline(answer);
```

12 Current Limitations and Rules

- Source-level string values are not supported.
- User-defined boolean variables are not a separate type; booleans are represented as integer truth values.
- Function calls are statements. Return values are read from `retvar`.
- Dynamic list types are supported in function types as `List[T, rank]`, but the current compiler supports dynamic list parameters only for rank 1.
- Ordinary variable declarations use fully shaped list types such as `List[Int, 2, 3, 4]`.
- List indexes are zero-based and checked at runtime.
- A specification block cannot contain another specification block.
- `if ... fi` and `while ... od` are not terminated by semicolons.
- Future temporal verification requires MONA on `PATH`.
- The interpreter aborts after the configured maximum number of instructions.

13 Diagnostics

SelVeri reports failures as structured diagnostics with source spans when available. The implementation distinguishes parser, preprocessor, compiler, runtime, and verification errors. Common failures include:

- malformed source syntax or malformed specification syntax;
- nested or unclosed specification blocks;

- duplicate declarations in the same scope;
- use of undefined variables or undefined functions;
- assignment or argument type mismatches;
- list dimension or shape mismatches;
- out-of-bounds list indexing;
- failed verification conditions;
- missing MONA support for future temporal specifications.

Use `-debug` when diagnosing internal failures or inspecting final runtime state.